

串与模式匹配

失配之后，怎样复用已经确认的前缀信息

408 · 数据结构专题 DS-03

母问题：失配之后，哪些信息真的已经知道？

在一条较长主串中查找较短模式串时，每次失配之前其实已经证明了一件事：主串刚刚匹配成功的那一段后缀，与模式串开头的一段前缀完全相同。KMP 不把这条事实丢掉，而是继续问：**这段已经确认相等的后缀中，最长有多少字符还能直接作为模式串前缀继续使用？**

后文先用一个具体失配例子把这个问题跑通，再引入主串 T 、模式串 P 、下标和前缀函数。这样所有符号都有具体对象可指，不先靠抽象公式定义算法。

本册定位与归属边界

- **本册负责：**串的基本对象、朴素匹配、前缀函数、失配回退信息与 KMP 主循环。
- **调用：**数据结构总图的表示—操作—不变量—成本语言；底层字符序列只借用一般线性存储，不重新定义 DS01。
- **输出：**向算法专题提供线性时间模式匹配的可调用接口。
- **边界：**完整确定有限自动机、多模式匹配与文本索引属于后续扩展，不在本册展开。

目录

1	先从一次真实失配看懂 KMP 在做什么	3
1.1	朴素匹配为什么会重复工作	3
1.2	朴素匹配的两个指针与回退公式	3
2	前缀函数：模式自身到底有哪些信息可以复用	4
2.1	把 π 真正算一遍：ababaca	4
3	KMP 主循环：同一个文本字符为什么可以继续比较	5
3.1	KMP 的线性时间证明：把回退次数当作势能	5
3.2	教材中的 next 与 nextval：先固定状态语义	5
3.3	模式串为什么右移 $j - j'$ ：从起点坐标生成公式	6
3.4	输出契约：首个匹配与全部匹配不是同一个接受动作	7
4	代码：把两种扫描策略放在同一契约下	7
4.1	朴素基线：每个起点重新建立 matched	7
4.2	前缀函数：失配时只沿边界链回退	7
4.3	KMP：失配不倒退主串	8
4.4	教材编码：从前缀函数确定性生成 next/nextval	8
4.5	教材版主循环：让代码和手算使用同一组 (i, j) 状态	9

4.6	修正数组版：主循环不变，只替换失配表	10
4.7	测试：重复前后缀、教材编码与三种 KMP 等价性	11
5	复杂度与边界	13
5.1	串对象与存储边界	14
5.2	从单模式扫描到全文搜索：索引改变了成本归属	14
6	概念边界与做题控制协议	14
7	全科坐标与来源处理	15

1 先从一次真实失配看懂 KMP 在做什么

先固定两个对象： T 表示主串，即被搜索的较长字符串； P 表示模式串，即希望在主串中找到的较短字符串。模式匹配的输出版约是：返回 P 在 T 中第一次出现的起始位置，若不存在则返回 ‘npos’。本册代码另约定空模式返回 0。

同一个主串字符为什么不用回头重比

取

$$T = \text{abababaca}, \quad P = \text{ababaca}.$$

采用零起始下标。前 5 个字符已经匹配成功，此时准备比较的是主串字符 $T[5] = \text{b}$ 与模式字符 $P[5] = \text{c}$ ，因此在 $i = 5, j = 5$ 处失配。

失配之前已经知道

$$T[0 \dots 4] = P[0 \dots 4] = \text{ababa}.$$

而 ‘ababa’ 的末尾 ‘aba’ 与开头 ‘aba’ 相同，所以最后 3 个已经匹配的字符无需重新比较。模式状态可以直接从 $j = 5$ 缩短到 $j = 3$ ，主串位置仍保持 $i = 5$ 。下一次真正比较的是

$$T[5] = \text{b} \quad \text{与} \quad P[3] = \text{b},$$

这次比较成功。

因此 KMP 的关键不是先问 “模式串右移几格”，而是先问：**已经匹配成功的文本后缀中，最长有多少字符也等于模式前缀？**

现在再抽象。若当前准备比较 $T[i]$ 与 $P[j]$ ，并且前面已有 j 个字符匹配成功，则保持

$$T[i - j \dots i - 1] = P[0 \dots j - 1].$$

所以 i 表示当前主串位置， j 表示当前模式位置；在这个零起始状态模型里， j 同时也是已经匹配成功的字符数。失配后，KMP 只缩短 j ，尽量保留上式已经证明过的相等关系。

1.1 朴素匹配为什么会重复工作

朴素匹配枚举起点 $s = 0, 1, \dots, |T| - |P|$ ，在每个起点从左到右比较。若在第 j 个字符失配，已经比较过的前缀信息只用于判定当前起点失败，算法不会把它带到下一起点。最坏比较次数可以达到 $\Theta((n - m + 1)m)$ ，其中 $n = |T|, m = |P|$ 。

例如文本 $T = \text{aaaaab}$ 、模式 $P = \text{aaab}$ 时，多个起点都会先匹配若干个 ‘a’ 再失配。朴素算法反复重新确认这些 ‘a’；KMP 则把模式自身的重复结构预先记录下来，让失配后的新状态直接继承已经确认的信息。

1.2 朴素匹配的两个指针与回退公式

朴素匹配常用文本指针 i 和模式指针 j 。当 $T[i] = P[j]$ 时同时前进；失配且 $j > 0$ 时，当前起点已经失败，模式回到 0，文本应回到 “该起点后的下一个字符”：

$$i \leftarrow i - j + 1, \quad j \leftarrow 0.$$

如果代码采用 “比较失败后循环末尾再统一执行 $i++$ ”，就会把同一语义写成 $i \leftarrow i - j + 2$ 。两个公式不是不同算法，而是指针更新时机不同；判断是否正确要看下一轮比较的文本位置，而不是死记加一的数值。

朴素匹配的不变量是：进入每个起点时， i 指向该起点， $j = 0$ ；循环结束时，已经比较的 $T[s..i - 1]$ 与 $P[0..j - 1]$ 相等，或已找到第一个不等字符。它的最坏界来自很多起点重复确认同一主串片段，而不是来自“回退”这个词本身。用朴素匹配作为基线，可以明确 KMP 真正删除的是“重新建立匹配前缀”的重复工作。

2 前缀函数：模式自身到底有哪些信息可以复用

先固定术语。前缀必须从字符串第一个字符开始，后缀必须在最后一个字符结束；真前缀和真后缀不能等于整个字符串。若一个字符串同时是真前缀和真后缀，本册称它为**边界**。例如‘ababa’的真前缀有‘a, ab, aba, abab’，真后缀有‘a, ba, aba, baba’，共同部分为‘a, aba’，所以最长边界是‘aba’，长度为 3。

为了不和匹配阶段的主串下标 i 混淆，这一节统一用 r 表示“当前正在构造前缀函数的模式串位置”。对模式前缀 $P[0..r]$ ，定义 $\pi[r]$ 为其最长边界的长度：

$$\pi[r] = \max\{k : 0 \leq k < r + 1, P[0..k - 1] = P[r - k + 1..r]\}.$$

计算 $\pi[r]$ 时，用 k 表示“当前候选边界的长度”。先令 $k = \pi[r - 1]$ ，意思是“先尝试把上一位置的最长边界再延长一个字符”。若 $P[r] \neq P[k]$ ，长度 $k + 1$ 的边界不可能成立。此时更短但仍可能成立的候选，必须是长度 k 这段前缀自己的边界，所以直接令

$$k \leftarrow \pi[k - 1]$$

继续尝试。若最终有 $P[r] = P[k]$ ，则令 $k \leftarrow k + 1$ ；若所有非空候选都失败，则保留 $k = 0$ 。这里特意不用字母 b 表示候选长度，避免与模式串中实际字符‘b’混淆。

因此构造不变量非常具体：处理完 $P[0..r]$ 后，数组第 r 项保存的就是该前缀最长边界的长度。每次回退都严格缩短候选长度，不会在已经排除的候选上反复比较。

2.1 把 π 真正算一遍：ababaca

继续沿用上面的模式串 $P = \text{ababaca}$ 。逐个加入新字符，并始终问“当前最长边界能否延长；不能时，这个边界自己的最长边界是谁”：

r	$P[r]$	旧候选长度 k	比较、回退与可复用事实	$\pi[r]$
0	a	无	单字符没有非空真边界	0
1	b	0	$b \neq a$ ，无法延长	0
2	a	0	$a = a$ ，空边界延长为‘a’	1
3	b	1	$b \neq P[1]$ ，‘a’延长为‘ab’	2
4	a	2	$a \neq P[2]$ ，‘ab’延长为‘aba’	3
5	c	3	$c \neq P[3]$ ：3 $\rightarrow$ 1 $\rightarrow$ 0，仍不等	0
6	a	0	$a = P[0]$ ，重新得到‘a’	1

因此 $\pi = [0, 0, 1, 2, 3, 0, 1]$ 。第 5 项的候选长度依次从 3 缩到 1、再缩到 0，是因为长度 3 的候选已经失败；后续仍可能成立的更短候选，只可能来自这段长度 3 前缀自己的边界链。

3 KMP 主循环：同一个文本字符为什么可以继续比较

为了让数学符号与代码变量分开，下面用 q 表示“已经匹配成功的字符数”；C++ 实现中的变量 ‘matched’ 保存的就是这个 q 。在准备处理当前主串字符 $T[i]$ 之前，保持不变量

$$T[i - q \dots i - 1] = P[0 \dots q - 1].$$

若 $T[i] \neq P[q]$ 且 $q > 0$ ，当前最长状态无法继续，但上式已经证明的文本后缀并没有消失，因此令

$$q \leftarrow \pi[q - 1]$$

改试更短的边界。注意此时同一个 $T[i]$ 仍未被消费，所以代码中的 ‘while’ 会继续比较，而不是移动主串下标。只有找到可以接上的模式位置，或所有非空候选都失败以后，外层扫描才进入下一个主串字符。

若当前字符能够接上，则 q 增加 1；当 $q = m$ 时，模式全部匹配完成，匹配起点为 $i + 1 - m$ 。对应到代码，就是 ‘matched++’ 与 ‘matched == pattern.size()’。

一次完整失配：文本不回头，模式状态缩短

继续使用 $T = \text{abababaca}$ 、 $P = \text{ababaca}$ 。当 $i = 5$ 时，前面已经匹配成功 5 个字符，因此 $q = 5$ 。当前真正比较的是主串字符 $T[5] = \text{b}$ 与模式字符 $P[5] = \text{c}$ ，二者失配。

由前缀函数可知 $\pi[4] = 3$ ，所以把已匹配长度从 5 缩短到 3。主串位置 i 不变，下一次真正比较的是同一个 $T[5] = \text{b}$ 与 $P[3] = \text{b}$ ，这次匹配成功，于是 q 增加到 4。整个过程中没有重新比较已经证实相等的 ‘aba’。

状态	迁移	必须保持的事实
$q = 0$	比较当前主串字符与 $P[0]$	没有可复用的非空模式前缀
$q > 0$ 且失配	沿前缀函数链缩短 q	不移动主串下标，保留已证实的后缀
匹配后	$q \leftarrow q + 1$	当前状态代表长度为 q 的相等前缀
$q = m$	返回 $i + 1 - m$	所有模式字符已连续匹配

3.1 KMP 的线性时间证明：把回退次数当作势能

主串下标 i 从不减小；已匹配长度 q 每次成功比较最多增加 1，每次失配回退都严格跳到更短的边界。一次失配可能沿前缀函数链回退多次，但每次回退都消耗此前积累的 q 。令势能 $\Phi = q$ ：成功比较最多让势能增加 1，回退则严格降低势能，所以整个主串扫描中的回退总量被此前的增长次数所限制。于是扫描时间为 $O(n)$ ，再加模式前缀函数的 $O(m)$ 构造，总时间为 $O(n + m)$ 。

3.2 教材中的 next 与 nextval：先固定状态语义

不同教材会把失配回退信息编码成不同数组；数组首项不是算法本体。为了和 408 常见手算口径建立一个可检验接口，本册额外固定一套零起始下标 + -1 哨兵的教材编码。设当前正在比较 $T[i]$ 与 $P[j]$ ，并且失配前已有

$$T[i - j \dots i - 1] = P[0 \dots j - 1].$$

于是 j 既是当前待比较的模式下标，也是已经匹配成功的字符数。若 $j > 0$ ，失配后仍可能复用的最长状态只取决于已匹配部分 $P[0 \dots j - 1]$ ，因此

$$\text{next}[0] = -1, \quad \text{next}[j] = \pi[j - 1] \quad (j > 0).$$

这就是“前缀函数右移一位、首位补 -1”的真正原因： $\text{next}[j]$ 回答的是“第 j 位失配以后怎么办”，所以读取的是失配前那 j 个字符的最长边界，而不是 $P[0 \dots j]$ 的最长边界。

若

$$P[j] = P[\text{next}[j]],$$

则已经知道 $T[i] \neq P[j]$ 时，回退后再拿同一个 $T[i]$ 与相同字符比较必然再次失败。于是可继续跳过这个无效状态：

$$\text{nextval}[j] = \begin{cases} -1, & j = 0, \\ \text{nextval}[\text{next}[j]], & j > 0 \text{ 且 } P[j] = P[\text{next}[j]], \\ \text{next}[j], & j > 0 \text{ 且 } P[j] \neq P[\text{next}[j]]. \end{cases}$$

这里 -1 只是哨兵状态，**不是长度为 -1 的前缀**。因此把 $\text{nextval}[j]$ 直接解释成“新对齐前缀长度”只在其非负时成立；更稳定的统一语言是“失配后的下一模式状态编码”。

从同一个 prefix 生成教材数组：aabaab

对 $P = \text{aabaab}$,

$$\pi = [0, 1, 0, 1, 2, 3],$$

故

$$\text{next} = [-1, 0, 1, 0, 1, 2], \quad \text{nextval} = [-1, -1, 1, -1, -1, 1].$$

例如 $j = 4$ 失配时， $\text{nextval}[4] = -1$ 表示没有值得用同一文本字符继续尝试的非负模式状态；标准哨兵主循环随后执行 $i += 1, j += 1$ ，下一次真正比较从模式下标 0 开始。

3.3 模式串为什么右移 $j - j'$ ：从起点坐标生成公式

设失配前模式起点为

$$s = i - j.$$

若失配后状态变为非负的 j' ，主串位置 i 不变，新模式起点为

$$s' = i - j'.$$

因此模式右移量不是 next 值本身，而是

$$\Delta = s' - s = j - j'.$$

在本节的修正数组口径下取 $j' = \text{nextval}[j]$ ，得到

$$\Delta = j - \text{nextval}[j].$$

当 $j' = -1$ 时，下一步哨兵迁移使真正的新起点成为 $i + 1$ ，故

$$(i + 1) - (i - j) = j + 1 = j - (-1),$$

同一公式仍成立。这个边界推导很重要：公式成立来自状态机的下一步行为，而不是把 -1 误当成某种前缀长度。

3.4 输出契约：首个匹配与全部匹配不是同一个接受动作

匹配接口还要区分“返回首个位置”和“枚举全部出现”。若允许重叠匹配，成功一次后不能把已匹配长度直接清零，而应令

$$q \leftarrow \pi[q - 1],$$

继续扫描下一个主串字符；例如模式‘aba’在‘ababa’中出现于 0 和 2。若只返回首个位置，接受状态即可立即结束。接口契约改变，状态复位策略随之改变。

4 代码：把两种扫描策略放在同一契约下

完整 C++17 实现位于 code/ds03_string_matching.hpp，测试位于 code/ds03_string_matching_test.cpp；正文代码块直接从真实源文件抽取。

4.1 朴素基线：每个起点重新建立 matched

```

1 inline std::size_t naive_search(std::string_view text, std::string_view pattern
  ) {
2   if (pattern.empty()) {
3     return 0;
4   }
5   if (pattern.size() > text.size()) {
6     return std::string_view::npos;
7   }
8   for (std::size_t start = 0; start + pattern.size() <= text.size(); ++start) {
9     std::size_t matched = 0;
10    while (matched < pattern.size() && text[start + matched] == pattern[matched]
11    ) {
12      ++matched;
13    }
14    if (matched == pattern.size()) {
15      return start;
16    }
17  }
18  return std::string_view::npos;
19 }
```

代码清单 1: 朴素匹配的起点枚举与局部比较

朴素版本的重要性在于提供复杂度基线：它没有错误，只是没有利用跨起点的结构信息。任何声称 KMP 更快的解释，都应能指出它省掉了哪类重复比较。

4.2 前缀函数：失配时只沿边界链回退

```

1 inline std::vector<int> prefix_function(std::string_view pattern) {
2   std::vector<int> prefix(pattern.size(), 0);
3   for (std::size_t pos = 1; pos < pattern.size(); ++pos) {
4     int border = prefix[pos - 1]; // 先尝试延长前一个位置的最长边界。
5     while (border > 0 &&
6            pattern[pos] != pattern[static_cast<std::size_t>(border)]) {
7       border = prefix[border - 1];
8     }
9     prefix[pos] = border + 1;
10  }
11  return prefix;
12 }
```



```

7      // 当前候选无法延长, 改试“当前边界自己的最长边界”。
8      border = prefix[static_cast<std::size_t>(border) - 1];
9  }
10     if (pattern[pos] == pattern[static_cast<std::size_t>(border)]) {
11         ++border;
12     }
13     prefix[pos] = border;
14 }
15 return prefix;
16 }

```

代码清单 2: 前缀函数的边界回退构造

代码变量 ‘border’ 只表示当前候选边界的长度。每次回退都跳到更短边界, 因此不会形成无限循环; 若新字符与候选前缀的下一字符相等, 再增长一次。‘prefix’ 数组保存的就是每个模式前缀的最长边界长度。

4.3 KMP: 失配不倒退主串

```

1 inline std::size_t kmp_search(std::string_view text, std::string_view pattern)
2 {
3     if (pattern.empty()) {
4         return 0;
5     }
6     const std::vector<int> prefix = prefix_function(pattern);
7     std::size_t matched = 0;
8     for (std::size_t i = 0; i < text.size(); ++i) {
9         while (matched > 0 && text[i] != pattern[matched]) {
10             matched = static_cast<std::size_t>(prefix[matched - 1]);
11         }
12         if (text[i] == pattern[matched]) {
13             ++matched;
14         }
15         if (matched == pattern.size()) {
16             return i + 1 - pattern.size();
17         }
18     }
19     return std::string_view::npos;
20 }

```

代码清单 3: KMP 主循环与完整模式接受

这段代码最容易误读的是 ‘while’。它不是“重新开始匹配”, 而是在不移动当前主串字符的前提下, 连续缩短可复用模式状态。代码变量 ‘matched’ 保存前文数学状态 q 。失配时, 代码读取 ‘prefix[matched - 1]’; 由于该数组项保存的正是 $\pi[q - 1]$, 所以它和前文的数学回退完全一致。

主串循环变量 ‘i’ 单调递增; 模式状态 ‘matched’ 才可能沿前缀函数链下降。于是预处理 $O(m)$ 加扫描 $O(n)$, 总时间 $O(n + m)$, 额外空间 $O(m)$ 。这不是“next 公式自动带来线性”, 而是由文本扫描不倒退、模式状态回退严格变短共同保证。

4.4 教材编码: 从前缀函数确定性生成 next/nextval


```

1 // 408 常见的零起始下标 + -1 哨兵编码:
2 // next[0] = -1, next[j] = prefix[j - 1] (j > 0)。
3 inline std::vector<int> next_sentinel(std::string_view pattern) {
4     if (pattern.empty()) {
5         return {};
6     }
7     const std::vector<int> prefix = prefix_function(pattern);
8     std::vector<int> next(pattern.size(), -1);
9     for (std::size_t j = 1; j < pattern.size(); ++j) {
10         // pattern[j] 失配时, 真正已经匹配完成的是 pattern[0..j-1]。
11         // 所以下一状态取这个已匹配部分的最长边界长度。
12         next[j] = prefix[j - 1];
13     }
14     return next;
15 }
16
17 // nextval 只优化“下一次要比较哪个模式状态”的编码;
18 // 它不改变前缀函数所描述的最长边界结构。
19 inline std::vector<int> nextval_sentinel(std::string_view pattern) {
20     if (pattern.empty()) {
21         return {};
22     }
23     const std::vector<int> next = next_sentinel(pattern);
24     std::vector<int> nextval(pattern.size(), -1);
25     for (std::size_t j = 1; j < pattern.size(); ++j) {
26         const int fallback = next[j]; // 本约定下 j > 0 时必有 fallback >= 0。
27         const std::size_t k = static_cast<std::size_t>(fallback);
28         if (pattern[j] == pattern[k]) {
29             // 当前字符和回退后的字符相同: 若当前比较失败, 回退后必然再次失败。
30             nextval[j] = nextval[k];
31         } else {
32             // 回退后的字符不同, 仍可能和当前主串字符匹配, 保留这个状态。
33             nextval[j] = fallback;
34         }
35     }
36     return nextval;
37 }

```

代码清单 4: 零起始、-1 哨兵的 next 与 nextval 生成

这两个函数故意建立在 prefix_function 之上: 先固定唯一数学对象, 再生成考试编码, 避免让 next 与 nextval 变成两套平行理论。以 aabaab 的 $j = 4$ 为例, $\text{next}[4] = 1$, 但 $P[4] = P[1] = a$; 如果当前文本字符已经与 $P[4]$ 的 'a' 失配, 再退到同样为 'a' 的 $P[1]$ 比较必然仍失败, 所以 $\text{nextval}[4]$ 继续取 $\text{nextval}[1] = -1$ 。

4.5 教材版主循环: 让代码和手算使用同一组 (i, j) 状态

```

1 // 与 408 手算完全同构的教材版 KMP:
2 // i 指向当前文本字符, j 指向当前模式字符; 普通失配只修改 j。
3 inline std::size_t kmp_search_sentinel(std::string_view text, std::string_view

```

```

    pattern) {
4   if (pattern.empty()) {
5       return 0;
6   }
7
8   const std::vector<int> next = next_sentinel(pattern);
9   std::size_t i = 0;    // 当前准备比较的主串 text 下标
10  std::ptrdiff_t j = 0; // 当前准备比较的模式串 pattern 下标
11  const std::ptrdiff_t m = static_cast<std::ptrdiff_t>(pattern.size());
12
13  while (i < text.size() && j < m) {
14      if (j == -1 || text[i] == pattern[static_cast<std::size_t>(j)]) {
15          // j == -1: 当前主串字符没有可继续尝试的模式状态;
16          // 字符相等: 当前比较已经成功。
17          // 两种情况都会消费当前 text[i], 因此 i 和 j 同时前进。
18          ++i;
19          ++j;
20      } else {
21          // 普通失配只缩短模式状态; 当前 text[i] 仍要继续参与比较。
22          j = next[static_cast<std::size_t>(j)];
23      }
24  }
25
26  return (j == m) ? (i - pattern.size()) : std::string_view::npos;
27 }

```

代码清单 5: 零起始、-1 哨兵的教材版 KMP 主循环

这里 i 指向当前文本字符, j 指向当前模式字符; 在普通失配分支中只有

$$j \leftarrow \text{next}[j],$$

没有 $i += 1$, 所以同一个 $T[i]$ 会继续与回退后的 $P[j]$ 比较。若最终回退到 $j = -1$, 下一轮进入哨兵分支, 同时令 $i += 1, j += 1$, 把模式首字符移到下一个文本起点。于是手算中的“ j 回退、 i 不动”与代码是一一对应的状态转移, 而不是口号。

前缀函数版和教材版的对应关系可以压成四项: 代码变量 ‘matched’ 与教材状态 j 都表示已经匹配成功的字符数; 代码查 ‘prefix[matched - 1]’, 教材查 ‘next[j]’, 两者都在寻找失配后的新模式状态; 普通失配时当前主串字符都不被消费; 代码中 ‘matched == pattern.size()’ 与教材中的 $j = m$ 都表示模式全部匹配成功。

4.6 修正数组版: 主循环不变, 只替换失配表

```

1  inline std::size_t kmp_search_nextval_sentinel(std::string_view text,
2                                                  std::string_view pattern) {
3      if (pattern.empty()) {
4          return 0;
5      }
6
7      const std::vector<int> nextval = nextval_sentinel(pattern);
8      std::size_t i = 0;
9      std::ptrdiff_t j = 0;

```

```

10  const std::ptrdiff_t m = static_cast<std::ptrdiff_t>(pattern.size());
11
12  while (i < text.size() && j < m) {
13      if (j == -1 || text[i] == pattern[static_cast<std::size_t>(j)]) {
14          ++i;
15          ++j;
16      } else {
17          // 与普通 next 版唯一的差别：直接跳过字符相同、已知必败的状态。
18          j = nextval[static_cast<std::size_t>(j)];
19      }
20  }
21
22  return (j == m) ? (i - pattern.size()) : std::string_view::npos;
23 }

```

代码清单 6: 使用 nextval 的教材版 KMP 主循环

‘nextval’ 版本没有改写 KMP 主循环的基本语义。匹配成功仍然同时推进主串与模式位置；普通失配仍然保持当前主串字符，只修改模式状态。区别只有失配表由 ‘next’ 换成 ‘nextval’：后者提前越过模式字符相同、已经能够判定必败的中间状态。因此它改变的是某些输入上的字符比较次数，不改变匹配结果，也不改变 $O(n + m)$ 的渐近上界。

4.7 测试：重复前后缀、教材编码与三种 KMP 等价性

```

1  void test_empty_and_boundary_patterns() {
2      assert(naive_search("abc", "") == 0);
3      assert(kmp_search("abc", "") == 0);
4      assert(kmp_search_sentinel("abc", "") == 0);
5      assert(kmp_search_nextval_sentinel("abc", "") == 0);
6      assert(naive_search("abc", "abcd") == std::string_view::npos);
7      assert(kmp_search("abc", "abcd") == std::string_view::npos);
8      assert(kmp_search_sentinel("abc", "abcd") == std::string_view::npos);
9      assert(kmp_search_nextval_sentinel("abc", "abcd") == std::string_view::npos);
10     ;
11     assert(naive_search("", "a") == std::string_view::npos);
12     assert(kmp_search("", "a") == std::string_view::npos);
13     assert(kmp_search_sentinel("", "a") == std::string_view::npos);
14     assert(kmp_search_nextval_sentinel("", "a") == std::string_view::npos);
15 }
16 void test_naive_and_kmp_agree() {
17     const std::vector<std::pair<std::string_view, std::string_view>> cases = {
18         {"abcabcabcd", "abcabcd"}, {"aaaaab", "aaab"}, {"mississippi", "issi"},
19         {"abcdef", "gh"}, {"abababab", "ababa"};
20     for (const auto& [text, pattern] : cases) {
21         const std::size_t expected = naive_search(text, pattern);
22         assert(expected == kmp_search(text, pattern));
23         assert(expected == kmp_search_sentinel(text, pattern));
24         assert(expected == kmp_search_nextval_sentinel(text, pattern));
25     }
26 }

```

```

27
28 void test_prefix_reuse_information() {
29     assert((prefix_function("ababaca") == std::vector<int>{0, 0, 1, 2, 3, 0, 1}));
30     assert((prefix_function("aaaa") == std::vector<int>{0, 1, 2, 3}));
31     assert((prefix_function("aabaaab") == std::vector<int>{0, 1, 0, 1, 2, 2, 3}));
32 }
33
34 void test_408_next_and_nextval_encodings() {
35     assert((next_sentinel("aabaaab") == std::vector<int>{-1, 0, 1, 0, 1, 2, 2}));
36     assert((nextval_sentinel("aabaaab") == std::vector<int>{-1, -1, 1, -1, -1, 2,
37         1}));
38     assert((next_sentinel("aabaab") == std::vector<int>{-1, 0, 1, 0, 1, 2}));
39     assert((nextval_sentinel("aabaab") == std::vector<int>{-1, -1, 1, -1, -1, 1}
40         ));
41     assert((next_sentinel("ababaaababaa") ==
42         std::vector<int>{-1, 0, 0, 1, 2, 3, 1, 1, 2, 3, 4, 5}));
43     assert((nextval_sentinel("ababaaababaa") ==
44         std::vector<int>{-1, 0, -1, 0, -1, 3, 1, 0, -1, 0, -1, 3}));
45 }
46
47 void test_repeated_mismatch_does_not_lose_text_progress() {
48     assert(kmp_search("ababababca", "abababca") == 2);
49     assert(kmp_search("aaaaaaaaab", "aaab") == 6);
50 }

```

代码清单 7: 边界输入、三种 KMP 对拍、前缀函数与教材数组

测试要求朴素匹配、前缀函数版 KMP、普通 ‘next’ 版 KMP 和 ‘nextval’ 版 KMP 在同一批主串/模式串上返回相同的首个位置，同时直接检查前缀函数与教材数组的数值，并覆盖空模式、模式过长、完全无匹配和失配后仍有答案的情形。

```

1 std::size_t count_sentinel_comparisons(std::string_view text,
2                                         std::string_view pattern,
3                                         bool use_nextval) {
4     if (pattern.empty()) return 0;
5     const std::vector<int> failure =
6         use_nextval ? nextval_sentinel(pattern) : next_sentinel(pattern);
7
8     std::size_t i = 0;
9     std::ptrdiff_t j = 0;
10    std::size_t comparisons = 0;
11    const std::ptrdiff_t m = static_cast<std::ptrdiff_t>(pattern.size());
12
13    while (i < text.size() && j < m) {
14        if (j == -1) {
15            ++i;
16            ++j;
17            continue;
18        }
19

```

```
20     ++comparisons;
21     if (text[i] == pattern[static_cast<std::size_t>(j)]) {
22         ++i;
23         ++j;
24     } else {
25         j = failure[static_cast<std::size_t>(j)];
26     }
27 }
28 return comparisons;
29 }
30
31 void test_comparison_count_and_nextval_optimization() {
32     assert(count_sentinel_comparisons("abaabaabcabaabc", "abaabc", false) == 10);
33
34     const std::size_t next_count =
35         count_sentinel_comparisons("aaaacaaaaab", "aaaaab", false);
36     const std::size_t nextval_count =
37         count_sentinel_comparisons("aaaacaaaaab", "aaaaab", true);
38     assert(next_count == 15);
39     assert(nextval_count == 11);
40     assert(nextval_count < next_count);
41 }
```

代码清单 8: 字符比较次数与 nextval 优化的可执行验证

这里特意把“字符比较”定义为真正执行一次主串字符与模式字符的比较；进入 $j = -1$ 的哨兵分支只更新状态，不计字符比较。测试锁定 2019 年真题主串 ‘abaabaabcabaabc’、模式串 ‘abaabc’ 的比较次数为 10；同时用主串 ‘aaaacaaaaab’、模式串 ‘aaaaab’ 验证普通 ‘next’ 需要 15 次字符比较，而 ‘nextval’ 只需要 11 次，且二者匹配结果相同。

测试还枚举二字母表上长度不超过 6 的全部主串、长度不超过 4 的全部模式串，逐对验证三种 KMP 与朴素匹配的结果一致。穷举不是正确性证明，但会系统攻击空模式、重叠匹配、周期串、长边界链与无匹配状态。

```
1 clang++ -std=c++17 -Wall -Wextra -Werror -pedantic \
2     code/ds03_string_matching_test.cpp -o /tmp/ds03_test
3 /tmp/ds03_test
```

5 复杂度与边界

任务	朴素匹配	KMP	关键前提	失败边界
预处理模式	无	$O(m)$	前缀函数语义固定	空模式
扫描文本	$O((n - m + 1)m)$ 最坏	$O(n)$	主串下标单调前进	$m > n$
总额外空间	$O(1)$	$O(m)$	只保存模式信息	大模式内存预算
返回位置	首个合法起点	首个合法起点	接受后立即返回	无匹配返回 npos

不同教材的 ‘next’ 可能使用 “失配时跳到的下标” 或 “最长边界长度” 两种编码。做题时先写语义，再把题目下标转换到该语义；不能拿另一版本的初值和边界直接替换。

5.1 串对象与存储边界

串是字符受限的线性表，但题面中的 “字符个数” “存储长度” “子串长度” 仍需分别解释。长度为 n 的串有 $n(n+1)/2$ 个非空连续子串；若把空串也算入则再加 1。相同内容出现在不同位置时，在 “子串个数” 问题中是否去重必须由题设声明。

顺序串可以用固定数组、动态数组或长度字段加字符区表示；链式串可以把字符分块存储。表示改变的是随机访问、连接、扩容和额外空间成本，不改变子串与模式匹配的逻辑定义。若题目给出 “长度不计终止符”，C 风格终止字符只是实现边界；若问存储容量，则必须把终止符、块内空闲字符或长度字段的约定单独计入。

从长度语义翻译到教材 next

本册的前缀函数只维护 $\pi[r] = “P[0..r]$ 的最长边界长度”。若教材把 ‘next[j]’ 定义为 “模式状态 j 失配后跳到的状态”，零基长度状态下对应 $\pi[j-1]$ ($j > 0$)。若教材使用 -1 哨兵或一基下标，先画出 “已经匹配多少个字符”，再平移下标；数组首项长什么样不是算法本体。

5.2 从单模式扫描到全文搜索：索引改变了成本归属

对多篇文档做查询时，逐篇运行朴素匹配或 KMP 的成本为 “文档总字符数乘以查询次数”。KMP 只减少单篇扫描的失配浪费，并没有把文档集合变成可按词定位的结构。若查询重复出现，可以离线建立倒排索引：

$$\text{document} \rightarrow \text{token/term} \rightarrow \text{posting list of document IDs.}$$

此时查询阶段从扫描正文转为查索引，更新文档则要承担分词、词项维护和倒排表更新成本。倒排索引与 KMP 都是 “预处理换查询”，但保存的信息不同：KMP 保存模式自身的边界结构，倒排索引保存语料中的出现位置。不能因为两者都使用预处理，就把一个当成另一个的实现。

全文搜索还会引入大小写、分词、规范化、通配符和多模式查询等语义问题；这些属于扩展内容。408 题目若只给一个主串和一个模式串，优先调用本册的单模式契约，不凭 “工业搜索” 背景增加不存在的预处理。

6 概念边界与做题控制协议

概念 A	≠ 概念 B	真正区别	混淆后果
串	≠ 模式	被搜索的文本对象 vs 约束模板	方向写反
边界	≠ 任意重复片段	必须同时是前缀和后缀	错误回退位置
前缀函数	≠ 匹配答案	模式内部信息 vs 文本中的出现位置	把预处理当搜索
失配回退	≠ 主串回退	只缩短模式状态	复杂度退化
线性时间	≠ 每次比较 $O(1)$	总比较次数还需证明单调性	复杂度口号化

遇到匹配题，按 “固定返回契约 → 写朴素基线 → 找重复前后缀 → 固定 prefix/next 语义 → 画一次

失配回退 → 再报告复杂度”的顺序处理。忘掉公式后，只需复原：已经匹配了什么？失配时哪些后缀仍可能成为模式前缀？模式状态能否缩短而不重扫文本？

7 全科坐标与来源处理

全科坐标：前缀 / 失配 / 复用

DS03 把线性序列上的“匹配失败”转化为模式内部的边界结构；它接收 DS01 的顺序访问成本语言，并向算法专题输出 KMP 的线性扫描接口。自动机、多模式和文本索引属于后续扩展，不改变本册的唯一知识归属。

本册吸收归档总册对 KMP 的核心表述“已匹配前缀压缩为失败状态”，并将 `next` 下标差异明确为表示约定问题。代码和测试只验证本册固定的 `prefix-length` 语义，避免把不同教材的数组编码混作第二套稳定真相。